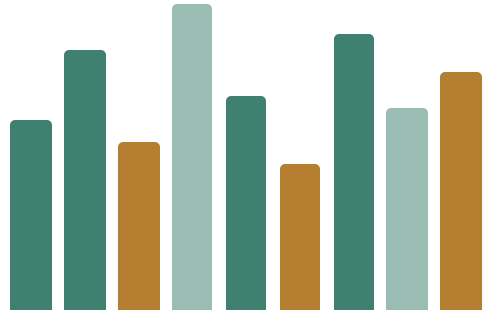
POC KICK-OFF 30 MD · 6 WEEKS

Recommendations that help readers find their next book

A 30-man-day proof of concept to test one belief: that recommendations will help you sell more books. What we will prove, how, and what it takes to run it in production.



PREPARED BY
Petr Dvořáček
Solution lead · AI Consultant / Architect

FOR
Online Book Shop · leadership & e-commerce team

You don’t need a recommendation engine. You need readers to find their next book, and buy it

The opportunity

Three levers on revenue

- **Discovery:** a relevant next book instead of a dead end
- **Conversion:** more visits end with a purchase
- **Basket:** one more book per order (series, same author)

The PoC must show whether these levers move, and whether it pays to build.

What your data already told us

- **Most readers are new or occasional**, so “similar books” on the book page, which works for everyone, is the biggest lever.
- **On your ratings, a simple model ranked 3.5× better than the bestseller list** and used 77 % of the catalogue, not 0.1 %. The PoC re-tests it on your order history.
- **Ratings alone cannot prove sales lift.** Orders and an A/B test can.

What we propose

30 MD · 6 weeks · 3 decision gates

- Book page + “picked for you”, on your data
- Proof: beats bestsellers offline and in a blind test by your experts
- Production design, business case, go / no-go for an A/B pilot
- Not in 30 MD: live integration and running the A/B test (slide 06)

I lead the solution team. Before this meeting we analysed the data you shared and tested five approaches, so this plan rests on evidence, not assumptions.

The PoC proves the engine is worth a live test. The live test proves it sells

Revenue tree

Revenue = sessions × conversion rate × average order value

LEVER	HOW RECOMMENDATIONS MOVE IT	KPI
Discovery	Shoppers find a relevant next book instead of leaving	CTR
Conversion	More sessions end with a purchase	CR
Basket	One more book per order (series, same author)	AOV
Catalogue	Sales spread beyond the top bestsellers	Coverage

North-star KPI for the pilot: **revenue per session**. CTR and add-to-cart explain why it moved.

PoC success criteria

Proposal, to agree today

- Model beats the best baseline on your held-out orders, **significantly** (95 % CI) and by **≥ 20 %** NDCG@10
- **≥ 70 %** of sampled lists rated “would show this” by your category experts
- Integration path and run cost agreed with IT
- Business case built on your own traffic and conversion numbers

Why 20 %: deliberately conservative (+250 % on your ratings). The final bar is checked against the break-even uplift in the business case.

Offline ≠ online. The PoC shows the model is good. Only a live A/B test shows it sells.

We have who-read-what, but no purchases, dates or genres

What the data you sent contains

TABLE	ATTRIBUTES	ROWS
Books	ISBN, title, author, year, publisher, cover URL	271 k
Ratings	user, ISBN, rating 0–10	1.15 M
Users	ID, location, age (40 % missing)	279 k

Ratings / interactions	yes
Purchases, prices, orders	missing
Dates, sessions, clicks	missing
Genres, descriptions, stock	missing

56 %
of users have **one** interaction; only 22 % have 5+

62 %
of ratings are **0**: interacted, not scored

4.4
avg. rating of the most-interacted book (a giveaway)

15
ISBNs for one Harry Potter book

Assumptions we make, and how we treat bad data

- **Any interaction = interest**; ratings 1–4 are not interest (1.7 % of scores)
- **One book = one work**: 270 k ISBNs merged into 227 k works
- **Dropped and counted:** 10 % ratings without metadata, malformed ISBNs, 46 bulk-upload accounts
- **Not used:** age and location. Weak signal, and sensitive under GDPR

My take — These ratings are enough to build and test the engine, and we already did. They cannot prove sales lift. The risk of this PoC is the order data, not the algorithm, so gate G1 checks it first.

Source: your export (Books, Users, Ratings) · full analysis in the GitHub repo

Start simple: “readers who read this also read”, tested against bestsellers

APPROACH	ON THIS DATA	NDCG@10	COVERAGE	ROLE
Popularity count / quality-weighted	Works, but same list for all	0.019	0.1 %	baseline, cold start
Content-based book attributes	Only title, author, publisher: too thin	–	–	after PoC
Item-kNN (BM25) collaborative	Best ranking, explainable, cheap	**0.066**	**77 %**	primary
ALS matrix factorisation	Close, but mostly bestsellers	0.054	8.5 %	challenger
Hybrid	Needs genres and descriptions	–	–	after PoC

Offline test on your ratings: 15.6 k readers × 29.4 k works, 20 % of each reader’s books hidden. Re-tested on orders in the PoC. Details: appendix A4.

How it works, in one breath

Two books are similar when the same readers read both. We discount very popular books and very heavy readers (BM25), so a mega-bestseller does not become “similar” to everything. Detail: appendix A3.

Cold start, solved per case

- **New visitor:** quality-weighted bestsellers
- **One book viewed:** its similar books
- **New title:** same author or series now; content model after PoC

My take — In 30 MD I would rather ship a model the business trusts than one nobody can explain. Complexity must beat the baseline to stay.

Try it: the prototype runs this exact model

PoC architecture: batch, offline and reproducible, with no changes to the live shop

1 · SOURCES

Order history
export, 2–3 years

Catalogue
ISBN, author, genre, price

Web events
views, if available

Ratings export
received, already analysed

2 · PREPARE

Ingest & validate
row counts, schema, join rates

Clean
merge editions → works, fix text, drop bulk accounts

Interaction matrix
customer × work, weighted signal

3 · MODEL & EVALUATE

Baselines
bestsellers, quality-weighted

Item-to-item similarity
primary model

ALS · challenger
+ content similarity for new titles

Evaluation harness
time split, metrics, segments

4 · OUTPUTS

Recommendation tables
top-20 per book & customer

Demo app
browse, compare, explain

Evaluation report
+ expert blind test

Stack · Python (pandas, implicit, scikit-learn), Databricks or your cloud workspace, MLflow for experiments, Git.

Why batch · Book tastes change slowly. Nightly lists are enough to prove value, and real-time adds cost before we know it pays.

Data stays with you · Work happens in your environment or a secured DataSentics workspace under a DPA; customer IDs pseudonymised.

What 30 MD buys, what it does not, and what comes next

In the PoC

- **Book page:** “readers also enjoyed”, for every visitor
- **Picked for you:** for customers with history
- **Fallback:** quality-weighted bestsellers
- Trained and validated on **your** orders and catalogue

Deliverables

D1	Data quality report	W1
D2	Clean data + pipeline code	W2
D3	Models + evaluation report	W4
D4	Demo app + expert blind test	W5
D5	Production design, run cost, business case, A/B plan	W6

Not in 30 MD, and why

- **Live integration** into the shop: needs your web team; we deliver the design and API contract
- **Running the A/B test:** needs integration and 4–6 weeks of traffic
- **Real-time recommendations:** nightly lists are enough to prove value
- **Cart and e-mail** placements
- **Business rules engine** (stock, margin): designed, not built
- **Catalogue enrichment** (genres, blurbs)

After a successful PoC

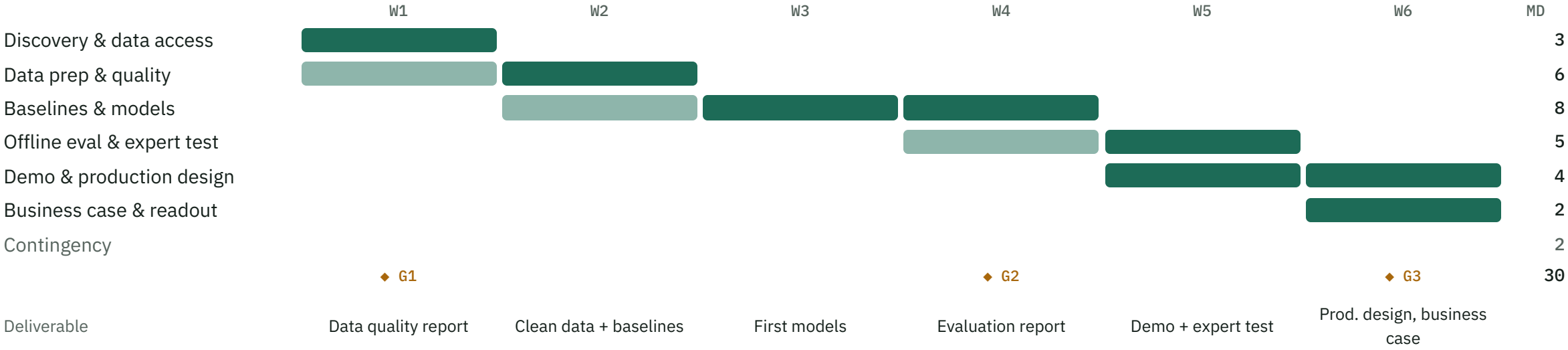
1. **A/B pilot** on the book page
2. **Business rules:** stock, margin, age, exclusions
3. **Hybrid model:** genres, blurbs, text embeddings for new titles
4. **Cart cross-sell** and personal newsletter
5. **Session-based** picks for anonymous visitors

Ordered by expected value for effort; each step is its own decision.

Run & maintain

Retraining and monitoring run automatically; a named owner reviews quality monthly and re-tunes quarterly. Run cost and effort are estimated in D5.

Six weeks, three decision gates, and clear dependencies on your side



Milestones & decision gates

G1	**End W1 · data good enough?** If not, we re-scope rather than burn days
G2	**End W4 · model beats the baseline** by the agreed margin?
G3	**End W6 · go / no-go** for the live A/B pilot

Dependencies on your side

- **Data export + DPA by day 3**, or the whole plan slides
- **2 category experts**: workshop in W1, blind test in W5
- **Web / IT lead** for integration design in W5
- **Sponsor** at Friday steering and each gate

2 MD contingency held by the solution lead, released only through steering

A small DataSentics team, the right people on your side, and a weekly rhythm

DataSentics · 30 MD

AI Consultant / Architect `me · solution lead`	Scope, architecture, client contact, decisions	7
Data Scientist	Models, evaluation, expert test	12
Data Engineer	Access, ingestion, cleaning, quality	6
ML Engineer	Demo app, production design, cost	5

Online Book Shop

Executive sponsor	Owns the goal, decides at gates	~2 h/wk
Product owner	Priorities, accepts deliverables	½ day/wk
Data / IT owner	Exports, access, security	W1
Category experts (2)	Business context, blind test	W1, W5
Web lead + DPO	Integration path, GDPR	ad hoc

How I run it

- **Weekly loop:** Monday plan with the PO, daily stand-up, Friday demo + 30-min steering
- **Budget in sight:** MD burn vs. plan every Friday
- **One owner** and a definition of done per deliverable
- **Decision & risk log** shared with you; changes go via steering
- **Quality:** peer-reviewed code, tracked experiments, no metric without a baseline

Weekly detail: appendix A2 · risks: slide 11.

Four checks, each one harder to fool than the last

1 · Data

Is the input what we think it is?

- Reconcile totals with your finance numbers
- Automated checks: duplicates, join rates, outliers, bulk accounts
- Unit tests on every cleaning step

2 · Model, offline

Does it predict what people bought next?

- Train on the past, test on the last weeks of orders
- NDCG, recall and hit rate @10, always vs. baselines
- Coverage and novelty, results per customer segment

3 · People

Would a bookseller show this?

- Blind test: our list vs. bestsellers, rated by your experts
- Red-flag review: out-of-stock, wrong age group, duplicates

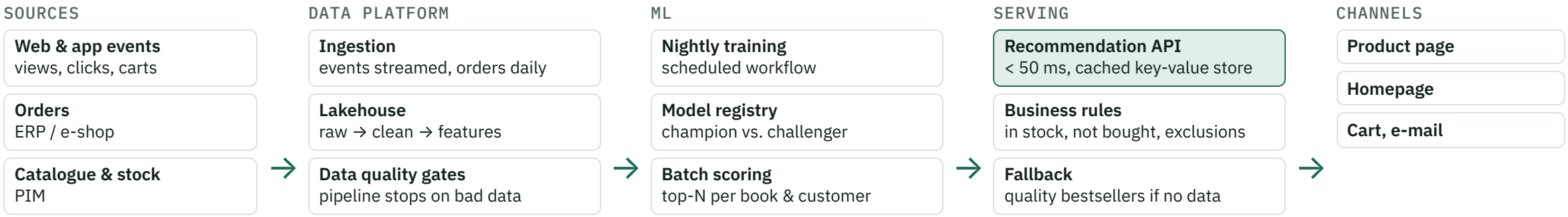
4 · Market · after PoC

Does it sell more?

- A/B test on the product page, 50 / 50 traffic
- Revenue per session; CTR, add-to-cart, AOV
- Guardrails: page speed, bounce rate. Duration from a power calculation on your traffic

House rule for the team: no metric is reported without its baseline, and every number can be reproduced from Git with one command.

In production: nightly models, a fast API with business rules, an A/B test on every change



PoC vs. production

	POC · 30 MD	PRODUCTION
Data	one-off exports	automated daily feeds + events
Training	run by hand, tracked	nightly job, champion vs. challenger
Serving	tables + demo app	API < 50 ms, rules, fallback
Proof	offline metrics + experts	A/B test, revenue dashboard
Run	Git, MLflow	CI/CD, drift alerts, named owner, rollback
Data protection	pseudonymised extract, DPA	consent check, retention, access control

Plugging into your shop

- A **API call** from the shop for each page: most flexible, needs a developer
- B **Platform plug-in** (e.g. Shopify, Magento, Shoptet) fed by our lists
- C **Nightly file feed** into the shop database: simplest start, no real-time

Which one depends on your systems: e-shop platform, order system / ERP, catalogue (PIM), web analytics, newsletter tool. We map them in week 1 and pick in W5.

My take: same code as the PoC, scheduled and monitored. Real-time session features added only if the pilot shows it pays.

What could go wrong, and what we need to learn from you

Open questions for you

QUESTION	WHY IT MATTERS
How many years of orders, with customer ID, date, price?	Without dates we cannot test “what they bought next”
What share of visits and orders is logged in?	Decides how far personal picks reach
What does a catalogue record contain: genre, blurb, stock?	New titles and business rules depend on it
Which shop platform, and who owns integration?	Sets the production path and its cost
Does consent cover personalisation? Can data leave your cloud?	Where we work and what we may use
Which KPI matters most, and what uplift pays back the investment?	Fixes the success bar and pilot length
Rules we must respect: own titles, publisher deals, age limits?	Shapes the business-rules layer

Top risks

RISK	MITIGATION
high Data access is late	Keep building on the ratings export meanwhile; G1
high Offline gain does not hold online	A/B pilot with guardrails before any rollout
med Order history thin or messy	Re-scope at G1 to the book page only
med No integration capacity	Name the owner today; agree API contract in W5
med Scope creep	Changes only via steering; backlog for later

Limitations we state openly: ratings cannot prove sales; without dates the offline test uses a random split, which flatters metrics; metadata is too thin for content models.

Five decisions today, and we start tomorrow

Decisions we need today

1. **Success criteria and gates** (slide 02, 07)
2. **First placement:** the book page
3. **Your people:** product owner, data owner, 2 category experts
4. **Where data is processed:** your cloud or our workspace under a DPA
5. **Steering slot:** Fridays, 30 minutes

What happens next

Day 1	We send the data export spec and DPA draft
Days 1–3	Workspace access, first data received
Week 1	2-hour workshop with your category experts
End W1	**G1: data quality readout (D1)**
Every Friday	Demo of something working + steering

In the meantime: try the prototype on the next page, built on your data.

→ TRY IT YOURSELF

Now see it work on real data

The prototype shows your shop with the recommendation engine switched on, built on your data. Open a book page, switch between a loyal reader and a new visitor, and compare with the bestseller list.

Open the prototype →

Metrics are not enough: a bookseller should nod at the lists

Reader looks at

The Hobbit · Tolkien

1. The Two Towers · Tolkien
2. The Return of the King · Tolkien
3. The Fellowship of the Ring · Tolkien
4. The Lord of the Rings · Tolkien
5. The Last Battle · C. S. Lewis

Series first, then a neighbouring fantasy classic.

Reader looks at

Bridget Jones's Diary · Fielding

1. Bridget Jones: The Edge of Reason · Fielding
2. Last Chance Saloon · Marian Keyes
3. Jemima J · Jane Green
4. Smart vs. Pretty · Valerie Frankel
5. Behind the Scenes at the Museum · Kate Atkinson

Learned the genre with no genre data at all.

Where it breaks

The Da Vinci Code · Dan Brown

1. Angels & Demons · Dan Brown
2. The Time Traveler's Wife · Niffenegger
3. All He Ever Wanted · Anita Shreve
4. Stupid White Men · Michael Moore
5. White Death · Clive Cussler

Mega-bestsellers are read by everyone, so their neighbours are noisy. Fix: genre metadata and business rules.

In the PoC these lists go to your category experts as a blind test: our list vs. the bestseller list, without labels. Their verdict is a success criterion.

Real output of the item-kNN model on your ratings

Short loops, visible budget, decisions written down

Weekly rhythm

Mon	Plan the week with the product owner (30 min)
Daily	15-min team stand-up, blockers raised the same day
Thu	Internal technical review of results and code
Fri	Demo of something working + 30-min steering with the sponsor

Control

- **MD burn vs. plan** reported every Friday; alert at 10 % over
- **One owner per deliverable**, clear definition of done
- **Risk & decision log** shared with the client
- **Change requests** go to steering, never straight into the sprint

Quality & team

- Code in Git, every change peer-reviewed
- Experiments tracked in MLflow, so results are comparable
- Pair work across data and model hand-offs
- I am the single point of contact; issues answered within one working day

Top risks: data access slips (we keep building on the ratings export) · weak order history (G1 re-scope) · offline result does not hold online (A/B pilot before any rollout) · no integration capacity on your side (agree owner at kick-off).

Item-kNN with BM25, step by step

1

Reader × book matrix

15.6 k readers × 29.4 k works. A cell is 1 if the reader interacted, weighted up to 3 for a high explicit rating.

2

BM25 weighting

Borrowed from search engines: a reader who read 1,000 books says less about each one, and a book everyone read says less about taste.

3

Book-to-book similarity

Compare the weighted reader columns of two books; keep the 200 most similar per book (K tuned on validation).

4

Recommend + explain

Book page: top similar books. Person: sum similarities over their history; the book that contributed most is the “because you read” reason.

Example

The Hobbit → *The Two Towers*: 111 readers read both. Next come the other Tolkien volumes, then other fantasy.

Why not deep learning?

Very sparse data (0.1 % density), no sequences or text to exploit, and a 30-MD budget. A neural model would cost more and be harder to explain, with no evidence it wins here.

Cost to run

Training takes seconds on a laptop CPU. Output is a lookup table, so serving is a key-value read.

Offline test on your ratings: setup and full results

Setup

- **Data:** cleaned 5-core, 536 k interactions
- **Split:** 20 % of each reader's books hidden at random (no dates exist)
- **Tuning:** on a second hidden 20 %, never on the test set
- **Question asked:** are the hidden books in the model's top 10?

Metrics in plain words

- **Hit rate@10:** share of readers with at least one hit
- **NDCG@10:** hits, rewarded more near the top
- **Coverage:** share of the catalogue ever recommended

On your data we switch to a time split: train on the past, test on the last weeks.

MODEL	P@10	R@10	NDCG@10	HIT@10	COVERAGE
Bestsellers (count)	0.010	0.020	0.019	8.1 %	0.1 %
Bestsellers (quality)	0.005	0.008	0.008	3.8 %	0.1 %
Item-kNN (cosine)	0.013	0.040	0.035	9.7 %	58.6 %
ALS	0.026	0.062	0.054	18.2 %	8.5 %
Item-kNN (BM25)	0.031	0.067	**0.066**	**19.2 %**	**76.5 %**

NDCG@10 by reader history size

BOOKS IN HISTORY	4–9	10–49	50+
Readers	8,220	5,688	1,734
Bestsellers (count)	0.012	0.016	0.061
Item-kNN (BM25)	0.045	0.071	0.148

The model wins most where it matters most: readers with short histories (3.8× the baseline).

Reproduce: python src/prep.py · python src/experiment.py (GitHub repo)